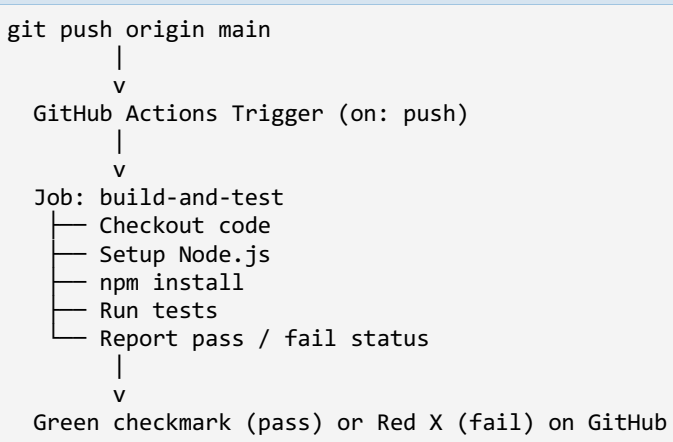
ADVANCED WEB DEVELOPMENT FRAMEWORKS (ITUE301)

Practical 12: CI/CD Pipeline with GitHub Actions

CO/PO	CO5 / PO3, PO5
Objective	To automate testing and deployment of the application using a CI/CD pipeline.
Prerequisites	• Practical 11 completed containerized full-stack application • GitHub account with the project repository already pushed • Basic understanding of YAML syntax
Theory Concepts (Self-Study Reference)	
Coursera Pointer	See Week 12 GitHub Actions Masterclass (Packt), Module 1 (workflow basics, triggers, jobs, steps) and Module 2 (CI/CD pipeline setup, automated testing on push).
Architecture / Diagram	
Diagram	
Problem Definition	
Problem Statement	• Set up a GitHub Actions workflow for the Task Management application that runs automatically on every push to the main branch. • The pipeline must install dependencies (npm install) as a defined step. • The pipeline must run at least one automated test and report a clear pass/fail status. • Demonstrate a failed pipeline caused by a deliberately broken test. • Demonstrate a fixed pipeline that passes after the test is corrected.
Key Questions / Analysis	• What is the difference between Continuous Integration and Continuous Deployment, and which one does this practical actually demonstrate? • Why does running tests automatically on every push catch problems earlier than testing manually before a release? • What would happen to the pipeline if the test file path were incorrect would the workflow fail at the same step as a genuine test failure?
Supplementary Problems	• Add a second job that runs a linter (e.g., ESLint) before the test job. • Configure the workflow to run only on pull requests targeting main, not on every direct push. • Add a status badge to the README that reflects the current pipeline status.
Key Skills Addressed	• Writing a GitHub Actions workflow YAML file • Configuring triggers, jobs, and steps for a CI pipeline

	• Writing at least one automated test that can run in CI • Reading and interpreting CI pipeline logs and pass/fail status
Learning Outcome	Students will be able to set up a basic CI pipeline using GitHub Actions that automatically installs dependencies and runs tests on every push, and will be able to diagnose a failing pipeline from its logs.
Lab Session Step-by-Step	

Steps	1. Install a basic testing library in the backend project:
	`npm install --save-dev jest supertest`
	2. Write at least one simple test (e.g., testing the GET /tasks endpoint returns status 200):
	`test('GET /tasks returns 200', async () => {`
	`  const res = await request(app).get('/tasks');`
	`  expect(res.statusCode).toBe(200);`
	`});`
	3. Create the workflow file at .github/workflows/ci.yml:
	`name: CI Pipeline`
	`on:`
	`  push:`
	`  branches: [main]`
	`jobs:`
	`  build-and-test:`
	`  runs-on: ubuntu-latest`
	`  steps:`
	`  - uses: actions/checkout@v4`
	`  - uses: actions/setup-node@v4`
	`  with:`
	`  node-version: 18`
	`  - run: npm install`
	`  - run: npm test`
	4. Commit and push the workflow file; confirm it appears under the Actions tab on GitHub.
	5. Deliberately break the test (change an expected value) and push again observe the red X / failed status.
	6. Fix the test and push again observe the pipeline turn green.
	7. Take screenshots of both the failed and passed pipeline runs for the lab journal.

Tools / Technology / Resources Required	• GitHub account and repository • GitHub Actions (no separate install required) • Jest or a similar Node.js testing library • Git CLI, Visual Studio Code
Total Hours	• Lab Session: 2 hours • Implementation: 1.5 hours • Testing / Modification: 0.5 hours • Total Engagement: 2 hours
Faculty Teaching Guide	
Guidance Notes	• Confirm GitHub access and internet connectivity in the lab before this session the entire practical depends on it. • Walk through the Actions tab UI together first so students know where to find pipeline runs and logs before debugging on their own. • Make the deliberate test failure a required step, not optional seeing a red X and reading the failure log is the core learning moment. • Keep the test itself trivial (e.g., a single status code check) the goal is understanding the pipeline mechanism, not testing depth.
Common Mistakes	• Placing the workflow YAML file in the wrong path (must be exactly .github/workflows/<name>.yml). • Incorrect YAML indentation, causing the workflow to fail to even register on GitHub. • Writing a test that always passes trivially (e.g., expect(true).toBe(true)), which defeats the purpose of the demonstration.

	• Forgetting to push the workflow file itself, then wondering why nothing appears under the Actions tab.

Troubleshooting Guide

Symptom	Likely Cause	Fix
Workflow does not appear under Actions tab	File not located at .github/workflows/ or has incorrect extension	Confirm the exact path .github/workflows/ci.yml and that it was committed and pushed
Workflow fails immediately with a YAML syntax error	Incorrect indentation in the YAML file	YAML is indentation-sensitive; use consistent 2-space indentation and avoid tabs
npm test fails with 'no test specified'	package.json test script not configured to run Jest	Set "scripts": { "test": "jest" } in package.json
Pipeline passes even with a broken test	Test was written incorrectly and does not actually assert the intended behavior	Review the test logic to confirm it fails when the code is genuinely broken

Student Assignment / Post Laboratory Work

Tasks	• Review the Theory Concepts section and the assigned Coursera module before next week. • Add a second automated test covering a different endpoint. • Add a status badge to the project README reflecting the current CI pipeline status.

GitHub Deliverables

Expected Repository Contents	• Same repository as previous practicals, updated with new commits • Working .github/workflows/ci.yml with install and test steps • Screenshots of both a failed and a passed pipeline run included in README or docs folder • At least one commit specifically for CI pipeline setup

Rubrics (Practical Evaluation / Viva)

Criteria (Marks)	Description
Conceptual understanding (2 Marks)	Explains what and why, not just what was done
Implementation (4 Marks)	Code written, runs, and is original
Correctness (2 Marks)	Output matches expected result
Reflection (1 marks)	Errors documented and explained
Lab file evidence (1 marks)	Completeness and presentation
Total: 10 marks	**Passing Threshold: 5 / 10 (50%)**